PRM393 — Final Assignment: One Complete System on Firebase

Oct 21, 2026 · @Lam Phuong

Join your Lab 2 design and Lab 3 app into **one complete system**: the mobile app, a small web admin for the other roles, one shared Firebase backend and one AI feature. The scope is moderate. Marks reward finishing, integrating and proving that the system works end to end, not new features. The grade is out of 10: the system 8, slides and demo video 2. A final report is required as evidence.

1. Assignment information

Item	Detail
Course	PRM393 — Mobile Programming
Builds on	Lab 2 (Figma, DESIGN.md, flutter-handoff.md) and Lab 3 (app, ARTEMIS tests, code review)
Tools	Flutter (mobile and web), Firebase CLI and Emulator Suite, Google ARTEMIS, GitHub
Submission	Repository link, web admin URL, report (PDF), slides (PDF), video link

2. Learning outcomes

1. Turn a design into a complete, deployed product that serves several roles.
2. Protect a shared cloud data model with role-based Security Rules, without a custom server.
3. Reuse one design system and one architecture across mobile and web.
4. Add an AI feature that is useful, respects user data and fails gracefully.
5. Prove that the system works end to end, and present it.

3. Task

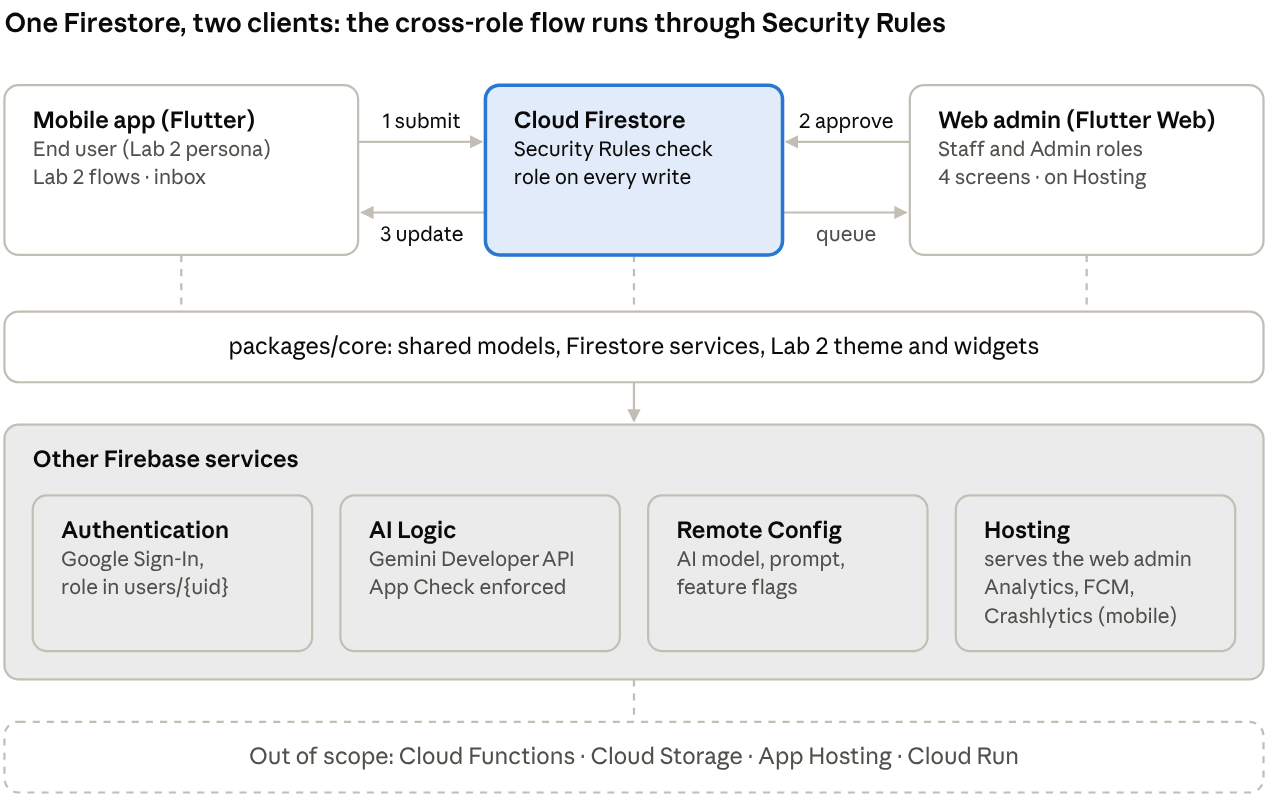
Part	Users	Minimum scope
Mobile app (Flutter)	Lab 2 persona	All Lab 2 screens and flows and all Lab 3 requirements finished; data on Firestore; Lab 3 review findings closed; inbox for announcements
Web admin (Flutter Web on Firebase Hosting)	Other roles, e.g. Admin, Staff	4 screens and an announcement form (section 7)
Backend (Firebase)	Both	Documented data model, role-based Security Rules with tests (section 6)
AI feature (Gemini)	Either	1 feature that helps a Lab 2 flow step (section 8)

Cross-role flow. At least one flow must cross roles and clients, for example: a student registers on mobile (*pending*) → the organizer approves on the web → the student's app updates in real time and shows an inbox message. Document it in `docs/cross-role-flow.md` (Appendix C) and show it live in the video.

Scope guard. New features earn no points; unfinished Lab 2/Lab 3 flows lose them. If your Lab 2 design has only one role, agree the web roles with the lecturer by [date].

Topic. Your Lab 2 / Lab 3 topic, unchanged.

4. System architecture



System architecture · cross-role flow in 3 steps

Both clients share one Firestore and one Dart package. The system has no custom server: logic runs in the clients, and Security Rules and App Check decide what each client may do. Use only the services in the diagram; Cloud Functions, Cloud Storage and App Hosting are out of scope. The Lab 3 PDF report is saved and shared on the device, with its metadata in Firestore.

5. Workflow

#	Stage	Output
1	Gap review of Lab 2 and Lab 3	`docs/gap-review.md`
2	Roles, data model, Security Rules and tests	`docs/data-model.md`, `firebase/`
3	Mobile app completed on Firestore	`mobile/`

#	Stage	Output
4	Web admin built and deployed	`web/`, live URL
5	Cross-role flow and AI feature	`docs/cross-role-flow.md`, `docs/ai-feature.md`
6	Tests, report, slides, video	`docs/test-report.md`, `report/`, `slides/`

6. Backend requirements

- **Data model** (`docs/data-model.md`, Appendix A): each collection with fields, types, which role may create, read, update and delete, and the screens that use it; allowed status transitions per role.
- **Roles:** `users/{uid}.role`. The first Admin is set in the console and named in the README. Only an Admin changes roles; nobody changes their own. Each client rejects accounts of the wrong role with a clear message.
- **Security Rules:** deny by default; no `if true` rules; validate data shape and status transitions. Test in the Emulator with at least one allowed and one denied case per role per collection (Appendix B).
- **Also:** App Check on Firestore and AI Logic; Remote Config and Analytics in both clients; Crashlytics on mobile.
- **Architecture:** Lab 3 MVVM in both clients; models and Firestore services in a shared package (`packages/core/`).

7. Web admin requirements

Screen	Must do
Sign-in	Google Sign-In; reject non-staff accounts
Management	List, search, create, edit, archive 1 entity the app reads
Review queue	Approve or reject items the app submitted, with a reason; the app updates in real time
Dashboard	⩾ 3 figures from aggregation queries and 1 chart

Screen	Must do
Announcement form	Writes to `announcements`; shown in the mobile inbox

Use the Lab 2 tokens and components with the wide-screen layout from your `flutter-handoff.md`. Every screen has loading, empty and error states. The deployed URL must open in a private window at the deadline. React or Vue is allowed if it uses the same tokens.

8. AI requirements (Gemini)

Add at least 1 Gemini feature, called through Firebase AI Logic (Gemini Developer API), that helps a step in a Lab 2 flow.

Level	Example	Grade
1. Text generation (streamed)	Summarise an abstract; draft an announcement	Partial
2. Structured output (JSON → Dart model, may include an image), confirmed by the user	Receipt photo → pre-filled expense form; auto-tag items in the review queue	Full
3. Function calling over the user's own data	"How much did I spend on food this month?"	Bonus

The feature must: use App Check; read model name, prompt and an on/off flag from Remote Config; handle quota and network errors so the flow still works without AI; send no personal data (names, e-mails, photos of people); label AI output and let the user edit it; log `ai_request`, `ai_result_accepted`, `ai_error`. Describe it in `docs/ai-feature.md` with three real inputs and outputs and one failure you handled.

9. Testing

Test	Minimum
Mobile E2E (ARTEMIS)	Every Lab 2 flow, the mobile steps of the cross-role flow and the AI feature; 3 runs each with pass rate

Test	Minimum
Security Rules (Emulator)	Allowed and denied case per role per collection; all pass
Web admin (checklist)	Main action of each screen, role rejection, empty and error states; a screenshot each
Cross-role flow	End to end on the deployed system, 3 times

Also run an AI code review on the new code: at least 3 findings with decisions. Evidence goes in `docs/test-report.md`.

10. Deliverables

Continue the Lab 3 repository, renamed `PRM393_Final_<ID>`:

```
├── README.md
├── packages/core/    mobile/    web/    e2e_test/
├── firebase/         (firebase.json, firestore.rules, indexes, remote-config
defaults, tests/)
├── docs/             (gap-review, data-model, cross-role-flow, ai-feature,
test-report)
├── evidence/
├── report/Final_<ID>.pdf
└── slides/Final_<ID>_slides.pdf
```

- **README:** roles, web URL, how to run each client and the emulators, demo accounts per role, versions, AI tools used, video link.
- **Report (10–15 pages):** changes since Lab 3, architecture, data model and rules, web admin, cross-role flow, AI feature, tests, code review, lessons learned, with evidence for every claim.
- **Slides (12–15):** persona, system diagram, each client, cross-role flow, AI feature, test results, lessons learned.
- **Video (8–12 min, live):** mobile flows; each web screen; the cross-role flow with both screens visible; the AI feature, including Remote Config switching it off; a test run.

11. Grading rubric (10 points)

Full, partial or zero per row. The report is required but not scored on its own; a criterion without evidence is not met.

Criterion	Pts	Full marks when
Mobile app	**2.0**	
Lab 2 and Lab 3 scope finished	1.25	Every Lab 2 flow and Lab 3 requirement works; gap review closed
Real data and inbox	0.75	All data on Firestore; announcements arrive in real time
Web admin	**2.0**	
Screens and actions	1.25	4 screens and announcement form work, with all states
Design system and deployment	0.75	Lab 2 tokens and components, wide-screen layout; URL works
Backend and integration	**2.0**	
Data model and Security Rules	1.0	Documented; deny by default; role-based; all rules tests pass
Cross-role flow	1.0	Works live, in real time, across both clients; documented
AI feature	**1.0**	Level 2 with Remote Config, App Check, fallback, no personal data (level 1: partial)
Testing and code quality	**1.0**	
Tests	0.5	ARTEMIS, rules and web checklist run with evidence
Architecture and code review	0.5	MVVM and shared package; ≥ 3 review findings with decisions
Slides and demo video	**2.0**	
Slides	1.0	All topics, evidence on every "it works" slide, readable

Criterion	Pts	Full marks when
Video	1.0	Every item in section 10, live, 8–12 minutes
Bonus (max +0.5, total ⩽ 10)	+0.5	AI level 3, or a second cross-role flow

12. Integrity and submission

- Git history must show work over time on top of Lab 3 (at least [N] commits over [N] days); in teams, every member's commits are visible.
- You may be asked for a 10-minute review: run the cross-role flow, change a Remote Config value, explain one rule and one AI decision. Marks can be adjusted after it.
- Undisclosed AI use, fabricated evidence or copied work fall under the university's academic integrity policy.

Checklist

- [ ] Gap review closed
- [ ] Data model documented; rules deny by default; rules tests pass
- [ ] Mobile app on Firestore with inbox
- [ ] Web admin: 4 screens + announcement form, deployed
- [ ] Cross-role flow works live and is documented
- [ ] AI feature with Remote Config, App Check and fallback
- [ ] Test evidence and ⩾ 3 review findings
- [ ] README, report, slides and video open in a private window

Appendix A — Data model template

Collection	Key fields (type)	Create	Read	Update	Delete	Used by screens
`users/{uid}`	`role` (string: user, staff, admin), `displayName` (string)	Self on first sign-in, role = user	Self; admin	Admin (role only)	—	Profile; web user list

Collection	Key fields (type)	Create	Read	Update	Delete	Used by screens
`events/{id}`	`title` (string ≤ 120), `startAt` (timestamp), `status` (draft, published, archived)	Staff	Everyone signed in (published only)	Staff	— (archive instead)	Mobile list and detail; web management
`registrations/{id}`	`eventId`, `uid`, `status` (pending, approved, rejected), `reason`	User (status = pending)	Owner; staff	Staff: pending → approved/rejected	Owner while pending	Mobile detail; web review queue

Appendix B — Rules test template

ID	Collection	Role	Operation	Expected	Result
R-01	registrations	user	create with status = approved	Denied	Pass
R-02	registrations	staff	update pending → approved	Allowed	Pass
R-03	users	user	update own `role`	Denied	Pass

Appendix C — Cross-role flow template

For each step: actor and client, screen, Firestore write (collection, fields), the rule that allows it, and what the other client shows. Draw it as a sequence diagram (Mermaid is fine) and add the measured time between the write on one client and the update on the other.